

```
import pandas as pd
idle_df = pd.read_csv('/content/idle.csv')
display(idle_df.head())
```

	Time (s)	Linear Acceleration x (m/s^2)	Linear Acceleration y (m/s^2)	Linear Acceleration z (m/s^2)	Absolute acceleration (m/s^2)	
0	0.001172	0.047525	-0.154775	-0.119342	0.201138	
1	0.011159	-0.073843	-0.054741	-0.134719	0.163091	
2	0.021147	-0.023077	0.004383	-0.072581	0.076287	
3	0.031134	-0.002025	-0.034365	-0.031386	0.046584	
4	0.041121	-0.018250	-0.053167	-0.025894	0.061889	

```
# Load walking.csv and situps.csv
walking_df = pd.read_csv('/content/walking.csv')
situps_df = pd.read_csv('/content/situps.csv')

print("Walking Data Head:")
display(walking_df.head())

print("\nSitups Data Head:")
display(situps_df.head())
```

Walking Data Head:

	Time (s)	Linear Acceleration x (m/s^2)	Linear Acceleration y (m/s^2)	Linear Acceleration z (m/s^2)	Absolute acceleration (m/s^2)	
0	-0.002110	-0.063357	0.012318	0.223961	0.233076	
1	0.007878	0.015780	0.032185	0.133945	0.138658	
2	0.017865	0.058337	0.045848	0.011061	0.075017	
3	0.027853	0.025792	0.050754	-0.028334	0.063593	
4	0.037840	-0.055451	0.091687	-0.034472	0.112559	

Situps Data Head:

	Time (s)	Linear Acceleration x (m/s^2)	Linear Acceleration y (m/s^2)	Linear Acceleration z (m/s^2)	Absolute acceleration (m/s^2)	
0	-0.000338	-0.475969	0.296619	0.050283	0.563078	
1	0.009649	-0.051216	0.263693	-0.213980	0.343430	
2	0.019637	0.043199	0.157093	-0.336843	0.374176	
3	0.029624	-0.010220	0.084323	-0.168470	0.188672	
4	0.039611	-0.132570	0.146062	-0.094046	0.218526	

```
import numpy as np

def extract_features(df, window_size_sec=1, sampling_rate_hz=100):
    # Calculate the 3D acceleration vector magnitude
    df['acceleration_magnitude'] = np.sqrt(
        df['Linear Acceleration x (m/s^2)']**2 +
        df['Linear Acceleration y (m/s^2)']**2 +
        df['Linear Acceleration z (m/s^2)']**2
    )
    samples_per_window = int(window_size_sec * sampling_rate_hz)
    features = []
    for i in range(0, len(df) - samples_per_window + 1, samples_per_window):
        window = df.iloc[i : i + samples_per_window]
        mean_acc = window['acceleration_magnitude'].mean()
        std_acc = window['acceleration_magnitude'].std()
        rms_acc = np.sqrt(np.mean(window['acceleration_magnitude']**2))

        features.append([
            mean_acc,
            std_acc,
            rms_acc
        ])
    return pd.DataFrame(features, columns=['mean_acc', 'std_acc', 'rms_acc'])
```

```
# Apply feature extraction to each dataframe
idle_features_df = extract_features(idle_df)
walking_features_df = extract_features(walking_df)
situps_features_df = extract_features(situps_df)

print("Idle Features Head:")
display(idle_features_df.head())

print("\nWalking Features Head:")
display(walking_features_df.head())

print("\nSitups Features Head:")
display(situps_features_df.head())
```

Idle Features Head:

	mean_acc	std_acc	rms_acc	
0	0.134209	0.053134	0.144247	
1	0.101117	0.035191	0.107008	
2	0.078118	0.031489	0.084166	
3	0.084528	0.031582	0.090180	
4	0.094724	0.038294	0.102100	

Walking Features Head:

	mean_acc	std_acc	rms_acc
0	0.596407	0.354726	0.693018
1	0.783181	0.265368	0.826492
2	0.935700	0.261891	0.971306
3	0.935890	0.358029	1.001395
4	0.970566	0.306980	1.017493

Situps Features Head:

	mean_acc	std_acc	rms_acc
0	0.955368	0.618083	1.136193
1	2.224229	1.068580	2.465287
2	2.012542	0.991899	2.241506
3	3.083219	2.506489	3.965590
4	1.488386	0.846076	1.709965

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Add labels to the feature dataframes
idle_features_df['label'] = 0 # Label for Idle
walking_features_df['label'] = 1 # Label for Walking
situps_features_df['label'] = 2 # Label for Sit-ups

# Combine all feature dataframes
all_features_df = pd.concat([
    idle_features_df,
    walking_features_df,
    situps_features_df
], ignore_index=True)

# Separate features (X) and labels (y)
X = all_features_df.drop('label', axis=1)
y = all_features_df['label']
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
# Scale features (important for neural networks)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Shape of X_train_scaled:", X_train_scaled.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of X_test_scaled:", X_test_scaled.shape)
print("Shape of y_test:", y_test.shape)
print("\nFirst 5 labels from y_train:\n", y_train.head())
```

Shape of X_train_scaled: (71, 3)
Shape of y_train: (71,)
Shape of X_test_scaled: (18, 3)
Shape of y_test: (18,)

First 5 labels from y_train:
5 0
11 0
13 0
31 0
30 0
Name: label, dtype: int64

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Define the MLP model
model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(3, activation='softmax')
])
# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Display model summary
model.summary()
# Train the model
history = model.fit(
    X_train_scaled, y_train,
    epochs=50, # You might adjust the number of epochs
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"\nTest Loss: {loss:.4f}")
print(f"\nTest Accuracy: {accuracy:.4f}")
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	256
dropout_2 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dropout_3 (Dropout)	(None, 32)	0
dense_5 (Dense)	(None, 3)	99

Total params: 2,435 (9.51 KB)
Trainable params: 2,435 (9.51 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/50
2/2 1s 248ms/step - accuracy: 0.2857 - loss: 1.1952 - val_accuracy: 0.3333 - val_loss: 1.1803
Epoch 2/50
2/2 0s 65ms/step - accuracy: 0.4643 - loss: 1.1020 - val_accuracy: 0.3333 - val_loss: 1.1168
Epoch 3/50
2/2 0s 68ms/step - accuracy: 0.5357 - loss: 1.0713 - val_accuracy: 0.4000 - val_loss: 1.0573
Epoch 4/50
2/2 0s 74ms/step - accuracy: 0.4643 - loss: 1.0507 - val_accuracy: 0.4000 - val_loss: 1.0022
Epoch 5/50
2/2 0s 66ms/step - accuracy: 0.4286 - loss: 1.0175 - val_accuracy: 0.4000 - val_loss: 0.9510
Epoch 6/50
2/2 0s 65ms/step - accuracy: 0.5714 - loss: 0.9318 - val_accuracy: 0.6667 - val_loss: 0.9048
Epoch 7/50
2/2 0s 70ms/step - accuracy: 0.5893 - loss: 0.8926 - val_accuracy: 0.6667 - val_loss: 0.8628
Epoch 8/50
2/2 0s 66ms/step - accuracy: 0.5536 - loss: 0.8941 - val_accuracy: 0.6667 - val_loss: 0.8266
Epoch 9/50
2/2 0s 65ms/step - accuracy: 0.6071 - loss: 0.8381 - val_accuracy: 0.6667 - val_loss: 0.7938
Epoch 10/50
2/2 0s 67ms/step - accuracy: 0.6250 - loss: 0.7893 - val_accuracy: 0.6667 - val_loss: 0.7643
Epoch 11/50
2/2 0s 64ms/step - accuracy: 0.6071 - loss: 0.7919 - val_accuracy: 0.6667 - val_loss: 0.7372
Epoch 12/50
2/2 0s 64ms/step - accuracy: 0.6250 - loss: 0.7726 - val_accuracy: 0.6667 - val_loss: 0.7125
Epoch 13/50
2/2 0s 70ms/step - accuracy: 0.6607 - loss: 0.7684 - val_accuracy: 0.6667 - val_loss: 0.6893
Epoch 14/50
2/2 0s 71ms/step - accuracy: 0.6429 - loss: 0.7115 - val_accuracy: 0.6667 - val_loss: 0.6681
Epoch 15/50
2/2 0s 78ms/step - accuracy: 0.6250 - loss: 0.7493 - val_accuracy: 0.6667 - val_loss: 0.6485
Epoch 16/50
2/2 0s 67ms/step - accuracy: 0.6964 - loss: 0.7010 - val_accuracy: 0.6667 - val_loss: 0.6304
Epoch 17/50
2/2 0s 65ms/step - accuracy: 0.6964 - loss: 0.6491 - val_accuracy: 0.6667 - val_loss: 0.6143
Epoch 18/50
2/2 0s 65ms/step - accuracy: 0.6429 - loss: 0.6606 - val_accuracy: 0.6667 - val_loss: 0.5996
Epoch 19/50
2/2 0s 66ms/step - accuracy: 0.6786 - loss: 0.6433 - val_accuracy: 0.6667 - val_loss: 0.5864
Epoch 20/50
2/2 0s 65ms/step - accuracy: 0.6607 - loss: 0.6228 - val_accuracy: 0.7333 - val_loss: 0.5747
Epoch 21/50
2/2 0s 66ms/step - accuracy: 0.6964 - loss: 0.6261 - val_accuracy: 0.7333 - val_loss: 0.5645
Epoch 22/50
2/2 0s 66ms/step - accuracy: 0.7143 - loss: 0.5972 - val_accuracy: 0.7333 - val_loss: 0.5554
Epoch 23/50
2/2 0s 69ms/step - accuracy: 0.6786 - loss: 0.5859 - val_accuracy: 0.7333 - val_loss: 0.5468
Epoch 24/50
2/2 0s 65ms/step - accuracy: 0.6429 - loss: 0.5850 - val_accuracy: 0.7333 - val_loss: 0.5385
Epoch 25/50
2/2 0s 67ms/step - accuracy: 0.7500 - loss: 0.5654 - val_accuracy: 0.7333 - val_loss: 0.5308
Epoch 26/50
2/2 0s 69ms/step - accuracy: 0.7321 - loss: 0.5735 - val_accuracy: 0.7333 - val_loss: 0.5234
Epoch 27/50
2/2 0s 68ms/step - accuracy: 0.7321 - loss: 0.5263 - val_accuracy: 0.7333 - val_loss: 0.5165
Epoch 28/50
2/2 0s 68ms/step - accuracy: 0.7143 - loss: 0.5198 - val_accuracy: 0.7333 - val_loss: 0.5098
Epoch 29/50
2/2 0s 65ms/step - accuracy: 0.8036 - loss: 0.5255 - val_accuracy: 0.7333 - val_loss: 0.5034
Epoch 30/50
2/2 0s 67ms/step - accuracy: 0.7679 - loss: 0.4966 - val_accuracy: 0.7333 - val_loss: 0.4972
Epoch 31/50
2/2 0s 71ms/step - accuracy: 0.7679 - loss: 0.4986 - val_accuracy: 0.7333 - val_loss: 0.4913
Epoch 32/50
2/2 0s 67ms/step - accuracy: 0.7679 - loss: 0.4670 - val_accuracy: 0.7333 - val_loss: 0.4857
Epoch 33/50
2/2 0s 74ms/step - accuracy: 0.8393 - loss: 0.5119 - val_accuracy: 0.7333 - val_loss: 0.4803
Epoch 34/50
2/2 0s 66ms/step - accuracy: 0.8214 - loss: 0.4925 - val_accuracy: 0.7333 - val_loss: 0.4748
Epoch 35/50
2/2 0s 67ms/step - accuracy: 0.8214 - loss: 0.4677 - val_accuracy: 0.7333 - val_loss: 0.4695
Epoch 36/50
2/2 0s 83ms/step - accuracy: 0.7857 - loss: 0.4665 - val_accuracy: 0.7333 - val_loss: 0.4644
Epoch 37/50
2/2 0s 72ms/step - accuracy: 0.8214 - loss: 0.4526 - val_accuracy: 0.7333 - val_loss: 0.4594
Epoch 38/50
2/2 0s 67ms/step - accuracy: 0.8393 - loss: 0.4370 - val_accuracy: 0.8000 - val_loss: 0.4546
Epoch 39/50
2/2 0s 64ms/step - accuracy: 0.8214 - loss: 0.4085 - val_accuracy: 0.8000 - val_loss: 0.4502
Epoch 40/50
2/2 0s 66ms/step - accuracy: 0.8393 - loss: 0.4409 - val_accuracy: 0.8667 - val_loss: 0.4458
Epoch 41/50
2/2 0s 66ms/step - accuracy: 0.8393 - loss: 0.4146 - val_accuracy: 0.8667 - val_loss: 0.4414
Epoch 42/50
2/2 0s 67ms/step - accuracy: 0.8571 - loss: 0.4067 - val_accuracy: 0.8667 - val_loss: 0.4366
Epoch 43/50
2/2 0s 82ms/step - accuracy: 0.8929 - loss: 0.3938 - val_accuracy: 0.8667 - val_loss: 0.4317
Epoch 44/50
2/2 0s 67ms/step - accuracy: 0.8214 - loss: 0.4242 - val_accuracy: 0.8667 - val_loss: 0.4269
Epoch 45/50
2/2 0s 70ms/step - accuracy: 0.9107 - loss: 0.3896 - val_accuracy: 0.9333 - val_loss: 0.4223
Epoch 46/50
2/2 0s 83ms/step - accuracy: 0.8571 - loss: 0.3817 - val_accuracy: 0.9333 - val_loss: 0.4179
Epoch 47/50
2/2 0s 70ms/step - accuracy: 0.8571 - loss: 0.3833 - val_accuracy: 0.9333 - val_loss: 0.4132
Epoch 48/50
2/2 0s 67ms/step - accuracy: 0.8571 - loss: 0.3807 - val_accuracy: 0.9333 - val_loss: 0.4086
Epoch 49/50
2/2 0s 68ms/step - accuracy: 0.8929 - loss: 0.3616 - val_accuracy: 0.9333 - val_loss: 0.4041
Epoch 50/50
2/2 0s 66ms/step - accuracy: 0.8571 - loss: 0.3470 - val_accuracy: 0.9333 - val_loss: 0.3996

Test Loss: 0.3145
Test Accuracy: 0.8880

```
import os
import tensorflow as tf
import numpy as np
```

```
# --- Variant A: Unquantized Baseline (Float32) ---
converter_float32 = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model_float32 = converter_float32.convert()
with open('har_model_float32.tflite', 'wb') as f:
    f.write(tflite_model_float32)
print(f"Float32 TFLite model size: {os.path.getsize('har_model_float32.tflite')} / 1024:.2f} KB")

# --- Variant B: Dynamic Range Quantization ---
converter_dr = tf.lite.TFLiteConverter.from_keras_model(model)
converter_dr.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model_dr = converter_dr.convert()
with open('har_model_dynamic_range.tflite', 'wb') as f:
    f.write(tflite_model_dr)
print(f"Dynamic Range TFLite model size: {os.path.getsize('har_model_dynamic_range.tflite')} / 1024:.2f} KB")

# --- Variant C: Float16 Quantization ---
converter_float16 = tf.lite.TFLiteConverter.from_keras_model(model)
converter_float16.optimizations = [tf.lite.Optimize.DEFAULT]
converter_float16.target_spec.supported_types = [tf.float16]
tflite_model_float16 = converter_float16.convert()
with open('har_model_float16.tflite', 'wb') as f:
    f.write(tflite_model_float16)
print(f"Float16 TFLite model size: {os.path.getsize('har_model_float16.tflite')} / 1024:.2f} KB")

# --- Variant D: Full Integer Quantization ---
converter_int8 = tf.lite.TFLiteConverter.from_keras_model(model)
converter_int8.optimizations = [tf.lite.Optimize.DEFAULT]
converter_int8.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
def representative_data_gen():
    for input_value in X_train_scaled.astype(np.float32):
        yield [np.array(input_value, dtype=np.float32).reshape(1, -1)]
converter_int8.representative_dataset = representative_data_gen

tflite_model_int8 = converter_int8.convert()
with open('har_model_full_integer.tflite', 'wb') as f:
    f.write(tflite_model_int8)
print(f"Full Integer TFLite model size: {os.path.getsize('har_model_full_integer.tflite')} / 1024:.2f} KB")
```

Saved artifact at '/tmp/tmp097bxy38'. The following endpoints are available:

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 3), dtype=tf.float32, name='keras_tensor_6')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  140711404754448: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752336: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752528: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404751952: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404750416: TensorSpec(shape=(), dtype=tf.resource, name=None)
Float32 TFLite model size: 11.60 KB
Saved artifact at '/tmp/tmp3kwdjh7v'. The following endpoints are available:
```

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 3), dtype=tf.float32, name='keras_tensor_6')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  140711404754448: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752336: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752528: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404751952: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404750416: TensorSpec(shape=(), dtype=tf.resource, name=None)
Dynamic Range TFLite model size: 6.00 KB
Saved artifact at '/tmp/tmp9wdja62q'. The following endpoints are available:
```

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 3), dtype=tf.float32, name='keras_tensor_6')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  140711404754448: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752336: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752528: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404751952: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404750416: TensorSpec(shape=(), dtype=tf.resource, name=None)
Float16 TFLite model size: 7.60 KB
Saved artifact at '/tmp/tmpn1o_9y82'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 3), dtype=tf.float32, name='keras_tensor_6')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  140711404754448: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752336: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752528: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404751952: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404752144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  140711404750416: TensorSpec(shape=(), dtype=tf.resource, name=None)
Full Integer TFLite model size: 7.70 KB
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/convert.py:863: UserWarning: Statistics for quantized inputs were expected, but not specified; continuing anyway.
  warnings.warn(
```

```
import time
import numpy as np

def evaluate_tflite_model(tflite_model_path, X_test_scaled, y_test):
    interpreter = tf.lite.Interpreter(model_path=tflite_model_path)
    interpreter.allocate_tensors()
    input_details = interpreter.get_input_details()
    output_details = interpreter.get_output_details()
    input_shape = input_details[0]['shape']
    output_shape = output_details[0]['shape']
    input_dtype = input_details[0]['dtype']
    predictions = []
    inference_times = []
    for i in range(len(X_test_scaled)):
        input_data = X_test_scaled[i:i+1].astype(np.float32)
        if input_dtype == np.int8:
            input_scale, input_zero_point = input_details[0]['quantization']
            # Quantize input with rounding
            input_data = np.round(input_data / input_scale + input_zero_point).astype(input_dtype)
            interpreter.set_tensor(input_details[0]['index'], input_data)
            start_time = time.time()
```

```
interpreter.invoke()
end_time = time.time()
inference_times.append((end_time - start_time) * 1000)
output_data = interpreter.get_tensor(output_details[0]['index'])
if output_details[0]['dtype'] == np.int8:
    output_scale, output_zero_point = output_details[0]['quantization']
    output_data = (output_data - output_zero_point) * output_scale
predictions.append(np.argmax(output_data))
accuracy = np.mean(predictions == y_test.values) * 100
avg_inference_time = np.mean(inference_times)
return accuracy, avg_inference_time
```

```
# Store model sizes (in KB) for the table
model_size_float32 = os.path.getsize('har_model_float32.tflite') / 1024
model_size_dr = os.path.getsize('har_model_dynamic_range.tflite') / 1024
model_size_float16 = os.path.getsize('har_model_float16.tflite') / 1024
model_size_int8 = os.path.getsize('har_model_full_integer.tflite') / 1024

accuracy_float32, time_float32 = evaluate_tflite_model('har_model_float32.tflite', X_test_scaled, y_test)
accuracy_dr, time_dr = evaluate_tflite_model('har_model_dynamic_range.tflite', X_test_scaled, y_test)
accuracy_float16, time_float16 = evaluate_tflite_model('har_model_float16.tflite', X_test_scaled, y_test)
accuracy_int8, time_int8 = evaluate_tflite_model('har_model_full_integer.tflite', X_test_scaled, y_test)

mem_reduction_dr = ((model_size_float32 - model_size_dr) / model_size_float32) * 100
mem_reduction_float16 = ((model_size_float32 - model_size_float16) / model_size_float32) * 100
mem_reduction_int8 = ((model_size_float32 - model_size_int8) / model_size_float32) * 100

results = pd.DataFrame({
    'Quantization Scheme': ['Baseline (Unquantized)', 'Dynamic Range', 'Float16 Quantization', 'Full Integer (PTQ)'],
    'Target Data Type': ['Float32', 'Weights int8, Act float32', 'Float16', 'Weights int8, Act int8'],
    'Model Size (KB)': [model_size_float32, model_size_dr, model_size_float16, model_size_int8],
    'Accuracy (%)': [accuracy_float32, accuracy_dr, accuracy_float16, accuracy_int8],
    'Memory Reduction (%)': [0.0, mem_reduction_dr, mem_reduction_float16, mem_reduction_int8],
    'Avg Inference Time (ms/window)': [time_float32, time_dr, time_float16, time_int8]})
print("\nUpdated Benchmarking Results Table:")
display(results.round(2))
```

Updated Benchmarking Results Table:
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration) for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)

	Quantization Scheme	Target Data Type	Model Size (KB)	Accuracy (%)	Memory Reduction (%)	Avg Inference Time (ms/window)
0	Baseline (Unquantized)	Float32	11.6	88.89	0.00	0.02
1	Dynamic Range	Weights int8, Act float32	6.0	88.89	48.27	0.00
2	Float16 Quantization	Float16	7.6	88.89	34.46	0.00
3	Full Integer (PTQ)	Weights int8, Act int8	7.7	88.89	33.65	0.00